Langages de script : python pour initiés

Cours 5 : Monitorer son programme + Visualiser des données

Rappels :

- Contrôle le 03/11 :

  Algorithmique / interview : /8

  Cours des 6 premières séances : /10

  Il faut revoir pour python : variables et types (str et float surtout)

  les conditions et les boucles

  les structures de données (listes et slicing, dictionnaires)

  les python tricks usuels

  les fonctions les plus utilisées de pandas

Interview

Interview 5 : regrouper les anagrames

Given an array of strings strs, group the anagrams together. You can return the answer in any order.

Example 1:

Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Explanation:

There is no string in strs that can be rearranged to form "bat". The strings "nat" and "tan" are anagrams as they can be rearranged to form each other. The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other. Example 2:

```python
def is_anagram(w1, w2) :

        if not  len(w1) == len(w2) : return False
        for letter in w2 :
            if letter not in w1 :
                return False
        return True

class Solution(object):
    def groupAnagrams(self, strs):

        anagram_lists = [[strs[0]]]
        for word in strs[1:]:

            found = False
            for group in anagram_lists :
                if is_anagram(word, group[0]) :
                    group.append(word)
                    found = True
                    break
            if not found :
                anagram_lists.append([word])

        return anagram_lists
```

2) 2 manières intelligentes de redéfinir is_anagram()

```
[ ]
```

```python
# Utiliser un compteur

from collections import Counter

def is_anagram(w1, w2):
    return Counter(w1) == Counter(w2)
```

```python
# Trier les chaînes !

def is_anagram(w1, w2):
    return sorted(w1) == sorted(w2)
```

- Complexité de certaines opérations élémentaires

Opération	Algorithme	Temporelle	Spatiale
Recherche			
Vérification si un objet est dans une liste non triée	Recherche linéaire	$O(n)$	$O(1)$
Vérification si un objet est dans une liste triée	Recherche binaire	$O(\log n)$	$O(1)$
Vérification si un objet est dans un dictionnaire	Hash map (moyenne)	$O(1)$	$O(n)$
Tri			
Tri d'une liste	Tri rapide (Quicksort)	$O(n \log n)$	$O(\log n)$
Accès			
Accès à un élément dans une liste	Accès indexé	$O(1)$	$O(1)$
Accès à un élément dans un dictionnaire	Hash map (moyenne)	$O(1)$	$O(n)$
Insertion			
Insertion à la fin d'une liste	$O(1)$	$O(1)$	
Insertion au début d'une liste	$O(n)$	$O(1)$	
Insertion dans un dictionnaire	Hash map	$O(1)$	$O(n)$
Suppression			
Suppression à la fin d'une liste	$O(1)$	$O(1)$	
Suppression au début d'une liste	$O(n)$	$O(1)$	
Suppression d'un élément dans un dictionnaire	Hash map	$O(1)$	$O(n)$

```python
def groupAnagrams( strs):

        wd_to_counter = {word: Counter(word) for word in strs}

        anagram_lists = [[strs[0]]]
        for word in strs[1:]:
            found = False
            for group in anagram_lists :
                if wd_to_counter[word] == wd_to_counter[group[0]]  :
                    group.append(word)
                    found = True
                    break
            if not found :
                anagram_lists.append([word])


        return anagram_lists
```

4) Optimal et élégant

```
1  def groupAnagrams( strs):
2
3          anagrams = {}
4          for word in strs:
5              sorted_word = ''.join(sorted(word))
6
7              if sorted_word in anagrams:
8                  anagrams[sorted_word].append(word)
9              else:
10                 anagrams[sorted_word] = [word]
11
12         return list(anagrams.values())
```

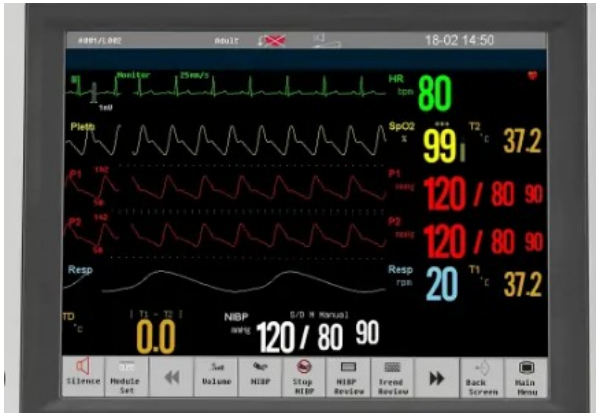


Monitorer son programme

Monitorer son programme

- Le fameux print(« here ») pour trouver l'origine des bugs de son code a ses limites. (programme énorme, milliers de fichiers traités en parallèle…) C'est pourquoi il existe d'autres techniques de monitoring intéressantes que l'on va étudier
- Monitorer c'est surveiller le programme et son exécution (qu'il s 'exécute bien, mais aussi se comporte correctement, soit rapide)
- Destiné à la fois au pogrammeur mais aussi à l'utilisateur pour comprendre ce qui se passe dans la boîte noire qu'est le programme

Python : les bonnes pratiques



1) Mesurer le temps d’exécution

2) Afficher des informations claires pour l’utilisateur :

   le logging et les barres de progression

3) la gestion des erreurs et des exceptions

1) mesurer le temps d’exécution

- Important pour améliorer un programme en production.
- Aussi repérer les problèmes : lenteurs dans le programme = suspect
- Permet de repérer quelles sections du code posent problème => remplacer une fonction en O(n3) par une fonction en O(n2) quand on a beaucoup de données peut diviser par 10 le temps !

  Il faut donc des outils pour repérer les parties lentes du code (bottleneck, goulot d’étranglement) et travailler ces sections

1) mesurer le temps d’exécution

- Python propose une solution simple nommé timers pour monitorer le temps d’exécution de parties de son code

```python
import time

def compter_la_classe(eleves):
    # Début du chronométrage
    debut = time.perf_counter()

    # Logique de la fonction : compter le nombre d'élèves
    nombre_eleves = len(eleves)
    print(f"Nombre d'élèves : {nombre_eleves}")

    # Fin du chronométrage
    fin = time.perf_counter()

    # Calcul du temps d'exécution
    temps_execution = fin - debut
    print(f"Temps d'exécution : {temps_execution:.6f} secondes")
    return nombre_eleves
```

1) mesurer le temps d’exécution

Problème : c’est long, il faut 3 lignes pour mesurer le temps d’exécution d’une seule fonction, et trouver comment stocker les valeurs si on le fait sur tout un code …

- On peut pour ça créer une classe Timer et des méthodes timer.start() timer.stop() qui feront ça de façon plus souple
- On peut créer une mémoire dans l’objet (dic) qui permette de stocker le nom de la fonction lancée

https://realpython.com/python-timer/ pour des idées d’implémentation

1) mesurer le temps d’exécution

Python got us covered

- python -m pip install codetiming

- from codetiming import Timer soit start et stop, soit context managre
- with Timer():
- compter_la_classe(eleves)
- Usage comme décorateur @Timer(name="decorator")

1) mesurer le temps d’exécution de son programme

- Un timer efficace est très utile pour monitorer la performance de chaque fonction quand on la code

  Mais parfois quand on travaille sur des programmes intenses, les petits delta sur des fonctions ont un grand impact, un timer classique ne repère pas les petites différences efficacement : il faut lancer la fonction des milliers de fois. Voir le module timeit

- Problème : quand le code devient vraiment compliqué, que l’on appelle certaines fonctions en boucle … même un bon timer ne permet pas de stocker toutes les informations de façon lisible

1) mesurer le temps d'exécution d'un programme

- Dans ce cas on parle plutôt de profiling du programme et peut utiliser des outils adaptés

  python -m cProfile -o nom_du_fichier

  souvent des outils en ligne de commande qui exécutent un code et affichent des statistiques sur le temps de chaque fonciton.
- Je recommande *snakeviz* pour des graphiques sympathiques

- Existe aussi pour faire du profiling de mémoire

2) afficher des informations claires pour l’utilisateur

- Logging : mémoriser des informations durant l’exécution d’un programme.
- Ça peut être la valeur d’une variable, les fonctions utilisées, le nombre d’appels, la taille d’un fichier , l’IP de l’utilisateur…
- Ensuite ces informations peuvent être affichées ou exportées

2) a) le logging

- import logging python a une solution plusieurs niveaux de sévérité

`DEBUG`	`logging.debug()`	Provides detailed information that's valuable to you as a developer.
`INFO`	`logging.info()`	Provides general information about what's going on with your program.
`WARNING`	`logging.warning()`	Indicates that there's something you should look into.
`ERROR`	`logging.error()`	Alerts you to an unexpected problem that's occured in your program.
`CRITICAL`	`logging.critical()`	Tells you that a serious error has occurred and may have crashed your app.

2) a) le logging

Modifier la configuration de base

logging.basicConfig(level=logging.DEBUG)

cet échelonnage en niveaux permet ensuite de définir quelles informations afficher à l'écran sans changer le travail de logging dans le code (le développeur affiche plus d info que pour l utilisateur…)

```
logging.basicConfig(
    filename="app.log",
    encoding="utf-8",
    filemode="a",
    format="{asctime} - {levelname} - {message}",
    style="{",
    datefmt="%Y-%m-%d %H:%M",
)
```

permet aussi de sauver les logs dans un fichier

2) a) le logging

La bonne pratique :

définir le bon niveau d’information pour chaque information affichée

* utiliser les self-documenting expressions pour stocker la valeur de certaines variables critiques

```
logging.debug("variable=%s", variable)
```

souvent pour des gros projets, les équipes définissent leurs loggers custom

2) b) les barres de progression

- Parfois quand on lance des boucles for avec des milliers d’appels et que le code est long, cela rassurerait l’utilisateur de savoir où on en est. Pour cela on peut afficher des barres de progression quand on fait des boucles

- Pour cela, python a tqdm
- cf (taqadum, تـــقدّم), progrès
- (te quiero demasiado).

2) b) les barres de progression

- pip install tqdm

  from tqdm(.auto) import tqdm

-

```
from tqdm import tqdm
    for i in tqdm(range(1000)):
    time.sleep(0.01)
```

  puis on wrappe l'itérable parcouru par tqdm. Marche dans les boucles for, les dicos, les listes par compréhension…

  paramètre desc, préciser la taille de l'objet...

2) b) les barres de progression

- Parfois dans la vie on a des cas plus complexes :
- Nested loops
- Programmes asynchrones
- On veut jouer avec pandas …

  et on aimerait quand même n’avoir qu’une seule barre de progrès pour tout

  La solution est de ne créer qu’une barre et de manuellement incrémenter la barre de progrès quand il faut

  Je n’en dis pas plus et vous laisserai jouer avec dans le TP 3 !

3) la gestion des erreurs

- Normalement un programme python termine dès qu'il rencontre une erreur. Mais ce n'est parfois pas ce qu'on voudrait !

  Ex : pipeline de traitement de données qui prend en entrée des fichiers words. Si qqn glisse par erreur un fichier jpeg il y aura une erreur. Mais on ne veut pas pour autant interrompre la chaîne de traitement et perdre tout le travail déjà fait.

- On peut donc gérer plus finement les erreurs pour préparer son programme à des malfonctions possibles et définir une réponse adaptée

3) la gestion des erreurs

Distinguer erreur vs exception (quand un code correct crashe lors de l’exécution)

Or toutes les erreurs ne sont pas les mêmes : division par 0, fichier n’existe pas, mémoire pleine… python donne plus d’informations sur la raison du crashe en distinguant plusieurs types d’exceptions.

3) gestion des erreurs

- Peut être une opportunité pour mieux contrôler son programme

  ex ; si on repère que le type de fichier est mauvais, inutile d'aller plus loin, on veut s'arrêter là => on *raise* une exception

  if eleve not in liste
- raise Exception(f" {eleve} n'a pas fait le contrôle!")

  print(note[eleve])

  pour généraliser ce genre de checkpoint on utilise le mot clé

  assert suivi d'une expression boléenne
- assert (eleve in liste), f" {eleve} n'a pas fait le contrôle!"

3) gestion des erreurs

- Problème : ça arrête le code. La meilleure manière de faire est d'utiliser un gestionnaire de contexte

  try : le code qu'on veut exécuter mais qui

  va potentiellement crasher

  except : ce qu'on fait si ça ne marche pas (lever

  une exception ou stocker l'erreur)

  else : ce qu'on fait ensuite si le code douteux

  n'a pas crashé

  finally : code exécuté quelle que soit l'issue



Source image; https://medium.com/@ccpythonprogramming/handling-exceptions-in-python-using-try-except-finally-4885e84a9491

3) gestion des erreurs

- On appelle ça l***'exception handling***

  => on prévoit tout ce qui peut aller de travers et on agit en conséquence

  utiliser try / except fait que le programme continue à tourner malgré l'erreur, il faut quand même conserver et traiter l'information que l'erreur a eu lieu.

  On peut raffiner en précisant quel type d'erreur on veut attraper (ex : file not found exception mais si le crash vient d'ailleurs on veut que le programme crashe)

  on peut enchaîner les except avec la façon de réagir à chaque type d'execption

3) gestion des erreurs

La bonne pratique dans un projet est de réfléchir à tout ce qui peut mal se passer dans votre code mais n’est pas de votre responsabilité (hardware, data envoyées…) et prévoir dans les sections critiques une gestion des exceptions prévisibles.

Vous pouvez créer vos propres exceptions particulières à la philosophie de votre programme pour les repérer et les traiter spécifiquement.

Buit-in vs custom exceptions

La gestion des erreurs

```python
class EleveAbsent(Exception):
    """Exception levée lorsqu'un élève est absent de la liste."""
    def __init__(self, nom):
        self.nom = nom
        super().__init__(f"L'élève {self.nom} est absent(e) de la liste.")

def verifier_presence(nom, liste_eleves):
    if nom not in liste_eleves:
        raise EleveAbsent(nom)
    print(f"L'élève {nom} est présent(e).")
```

D’autres sujets pour aller plus loin sur le monitoring

- Parfois avoir des fichiers de logs avec des valeurs n’est pas très interprétable. Il faut donc un moyen de visualiser l’évolution des résultats de son programme sur différents runs

  => création de dashboards. Ex librairie weight and biases

- On peut vouloir vérifier le bon comportement de son code sur des exemples canoniques pour être sur qu’une misa à jour de le détraque pas

  => création de fichiers de unit tests

Pour aller plus loin

- https://realpython.com/python-logging/
- https://www.datacamp.com/tutorial/tqdm-python
- https://realpython.com/python-exceptions/
-

Data visualisation

- L’objectif n’est pas de faire de beaux graphiques mais de représenter les données que l’on a d’une manière intéressante pour faciliter leur interprétation.
- Permet de communiquer avec des personnes qui n’ont pas un background technique / statistique
- Développement avec le marketing depuis les années 80, de nombreuses librairies existent avec leurs avantages et points faibles

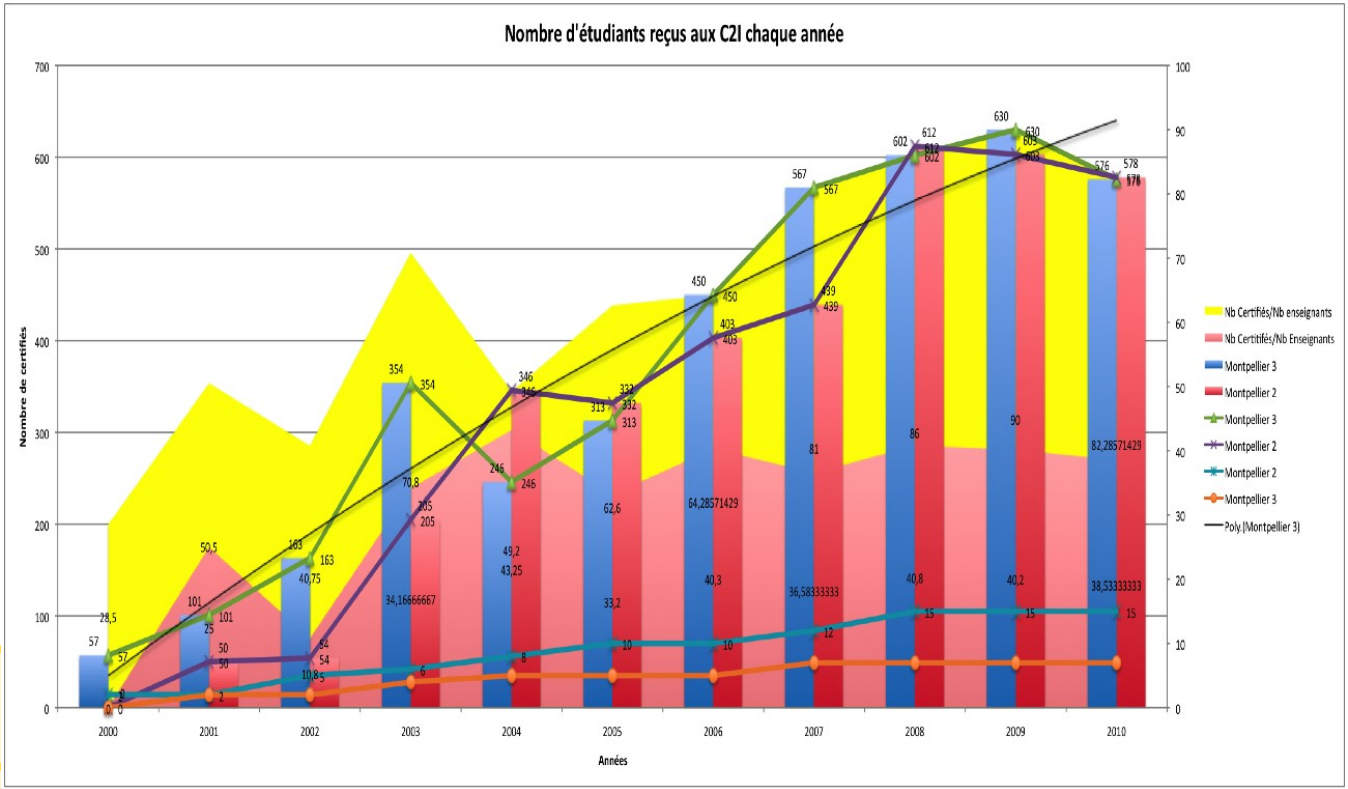
- 1) Les bonnes pratiques
- 2) Comment les construire avec les frameworks les plus connus ?
- 3) Quels types de graphiques existent ? Pour quoi faire ?
- 4) Quelques directions pour aller plus loin avec la dataviz

1) Les bonnes pratiques

* Sélectionner pour avoir un graphique lisible

- Savoir ce que l’on veut représenter et ne garder que les données intéressantes.

- Ne pas mettre trop d’informations sur le même graphique



1) Les bonnes pratiques

* Clarifiez le graphique

- Donner un titre clair
- Afficher les légendes sur l’axe des abscisses, des ordonnées
- Afficher une légende des couleurs et formes si besoin

* Choisir le type graphique adapté à la situation

2) Comment construire des graphiques avec matplotlib : une intro

La librairie la plus utilisée en python
From matplotlib import pyplot as plt…

Framework assez souple qui repose sur un objet appelé Figure
Un peu comme un tableau qui va contenir lui même d'autres éléments comme les Axes, les Legends …
Un peu comme dans un diaporama : on crée une slide et on met des objets dedans.

Cf Daniel Nelson, Data visualisation with Python

- ***Figure*** - The figure is what contains all of the other elements of the plot. You can think of it as the canvas that all of the elements of the plot are painted on.
- ***Axes*** - Plots have X and Y axes, with one variable located on the X-axis and one variable on the Y-axis.
- ***Title*** - The title is the description given to the plot.
- ***Legend*** - contains information regarding what the various symbols within the plot represent.
- ***Ticks*** - Ticks are small lines used to point to different regions of the graph, mark specific items, or delineate different thresholds. For example, if the X-axis of a graph contains the values 0 to 100, ticks may show up at 0, 20, 40, 60, 80, and 100. Ticks run along the sides, as well as the bottom, of the graph.
- ***Grids*** - Grids are lines in the plot's background that make it easier to distinguish where different values on the X and Y axes intersect.
- ***Lines/Markers*** - Lines and markers are what represent the actual data within a plot. Lines are typically used to graph continuous values, while markers/points are used to graph discrete values.

2) Comment construire des graphiques avec matplotlib : une intro

- Créer un graphique simple :

  #1 charger les données

  plt.plot(x, y) (x et y deux listes d'observations de MEME taille)

  #2 lancer l'affichage

  plt.show()

  C'est la fonction utilisée en #1 qui va changer le type de diagramme

2) Comment construire des graphiques avec matplotlib : ajouter légendes

```python
import pandas as pd
import matplotlib.pyplot as plt

# Charger les données Titanic depuis l'URL
url = "https://github.com/datasciencedojo/datasets/raw/master/titanic.csv"
df = pd.read_csv(url)

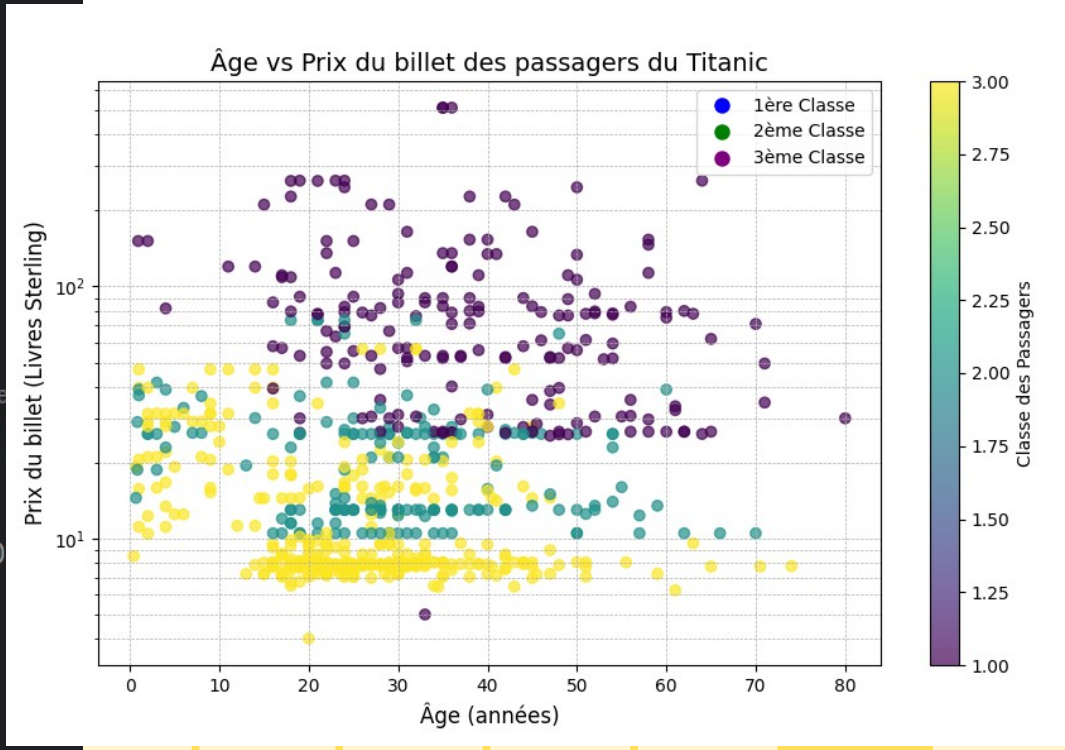
# Filtrer les données pour les passagers avec une valeur d'âge
df = df.dropna(subset=['Age'])

# Créer une figure et des sous-graphiques (subplots)
plt.figure(figsize=(10, 6))

# Générer un graphique de dispersion pour visualiser l'âge et le prix du bille
plt.scatter(df['Age'], df['Fare'], c=df['Pclass'], cmap='viridis', alpha=0.7)

# Ajouter un titre principal au graphique
plt.title( label: "Âge vs Prix du billet des passagers du Titanic", fontsize=14)

# Ajouter des labels pour les axes
plt.xlabel( xlabel: "Âge (années)", fontsize=12)
plt.ylabel( ylabel: "Prix du billet (Livres Sterling)", fontsize=12)
```



2) Comment construire des graphiques avec matplotlib : ajouter légendes

```python
# Ajouter une barre de couleur pour montrer la classe des passagers
cbar = plt.colorbar()
cbar.set_label('Classe des Passagers')

#Ajouter une légende manuelle pour les catégories de classe
from matplotlib.lines import Line2D
legend_elements = [
    Line2D( xdata: [0],  ydata: [0], marker='o', color='w', label='1ère Classe', markerfacecolor='blue', ma
    Line2D( xdata: [0],  ydata: [0], marker='o', color='w', label='2ème Classe', markerfacecolor='green', m
    Line2D( xdata: [0],  ydata: [0], marker='o', color='w', label='3ème Classe', markerfacecolor=
]
# Placer la légende sur le graphique
plt.legend(handles=legend_elements, loc='upper right')

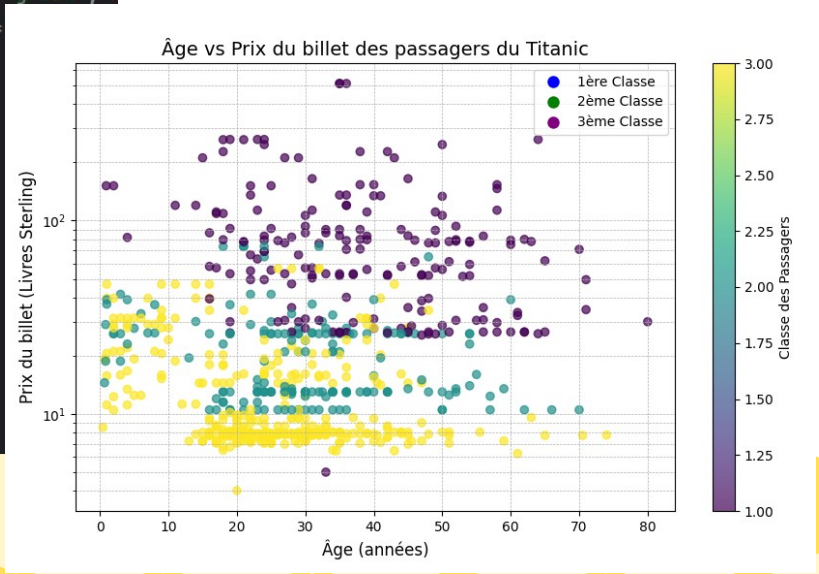
# Aj                                                    arithmique pour mieux visualiser)
plt.
# Af
plt.                                                    width=0.5)

# Afficher le graphique
plt.show()
```

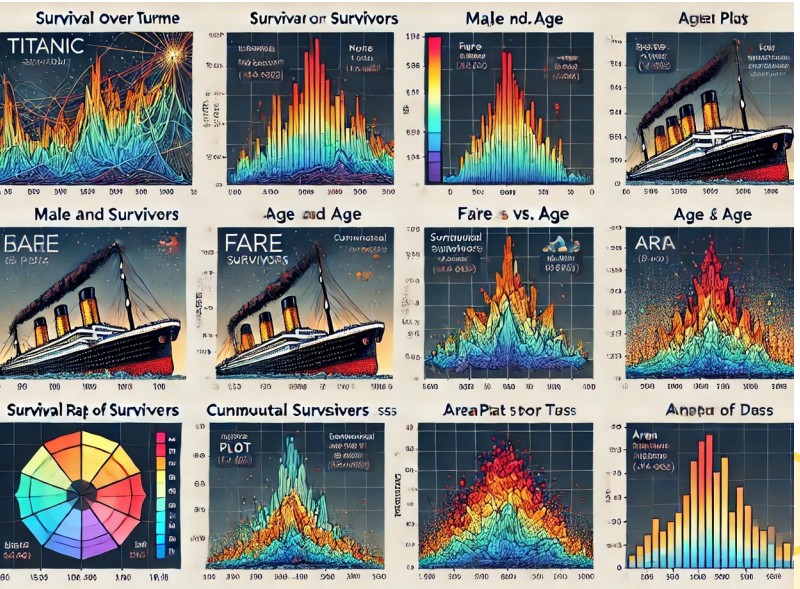
matplotlib.pyplot

```
@_copy_docstring_and_deprecators(Axes.legend)
def legend(*args: Any,
           **kwargs: Any) -> Legend
```

`legend(*args, **kwargs)` on matplotlib.org ↗



2) Les grands types de graphiques



Type de graphique	Description	Commande Matplotlib
Graphique linéaire	Représente des données continues avec des lignes reliant les points.	`plt.plot(x, y)`
Diagramme à barres	Utilisé pour comparer différentes catégories avec des barres.	`plt.bar(x, height)`
Histogramme	Montre la répartition d'un ensemble de données sous forme de barres.	`plt.hist(data, bins=n)`
Nuage de points (scatter)	Représente la relation entre deux variables en utilisant des points.	`plt.scatter(x, y)`
Graphique en secteurs (camembert)	Représente des parts d'un tout sous forme de secteurs.	`plt.pie(sizes, labels=labels)`
Boîte à moustaches (boxplot)	Montre la distribution statistique avec des quartiles et les valeurs extrêmes.	`plt.boxplot(data)`
Carte thermique	Représente les valeurs dans une matrice de données sous forme de couleurs.	`plt.imshow(data, cmap='hot')` ou `sns.heatmap(data)`
Graphique en aires	Représente des données accumulées sous forme de zones colorées.	`plt.fill_between(x, y1, y2)`

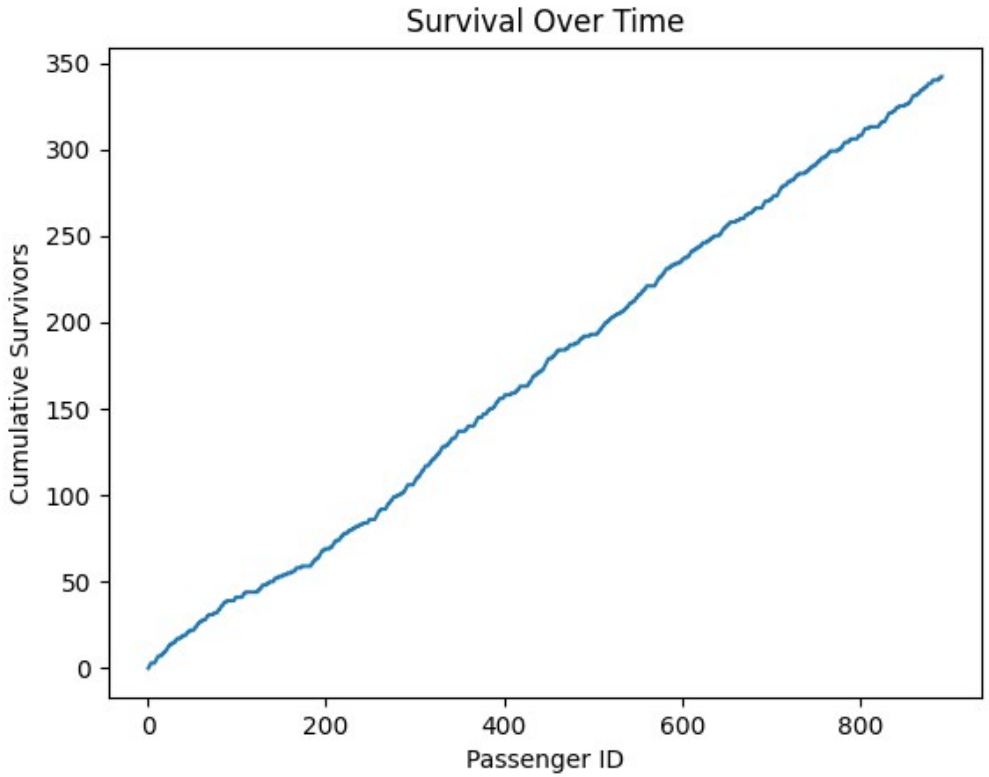
a) line plot

```python
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
url = "https://github.com/datasciencedojo/datasets/raw/master/titanic.csv"
df = pd.read_csv(url)

# Assume we have a 'Survival Time' (you might need to simulate it)
# Here, we'll just plot the number of survivors over time (by passenger ID)
survived_counts = df.groupby('PassengerId')['Survived'].sum().cumsum()

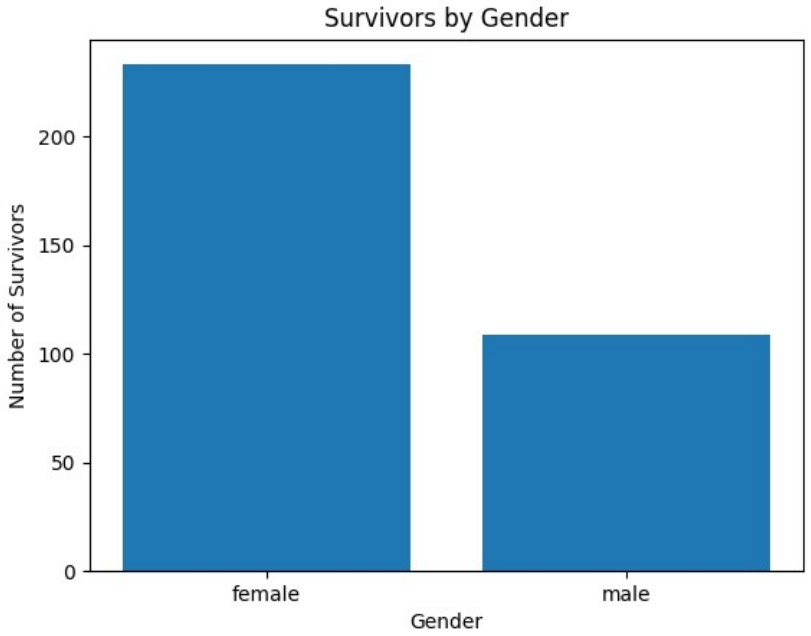
# Plot
plt.plot(survived_counts)
plt.title("Survival Over Time")
plt.xlabel("Passenger ID")
plt.ylabel("Cumulative Survivors")
plt.show()
```



b) Bar chart

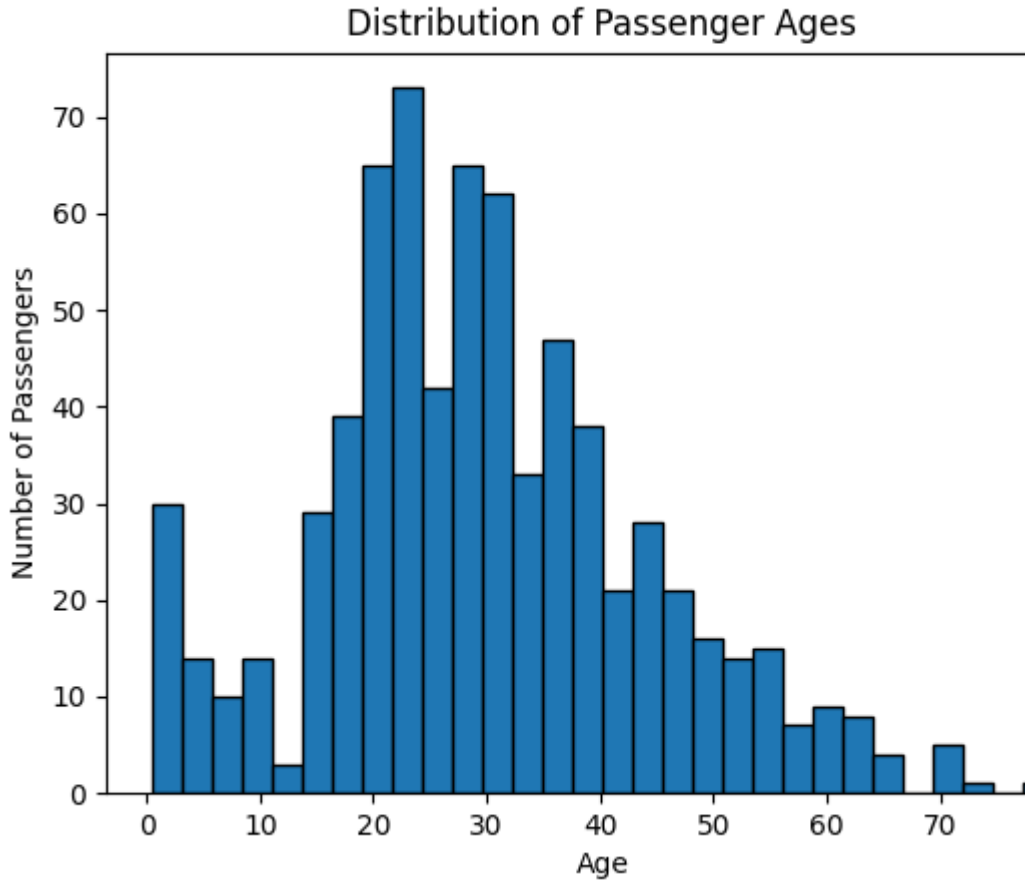
```
survivor_gender = df[df['Survived'] == 1]['Sex'].value_counts(

plt.bar(survivor_gender.index, survivor_gender.values)
plt.title("Survivors by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Survivors")
plt.show()
```



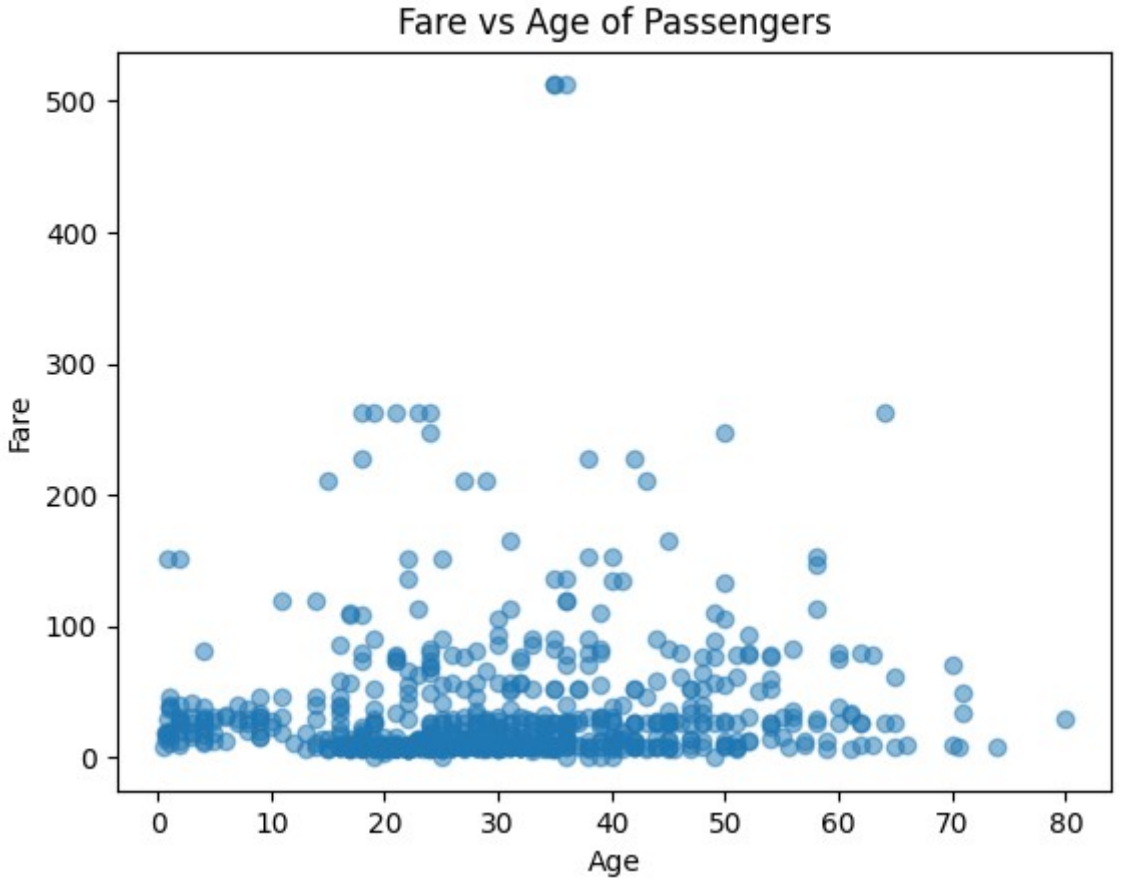
c) Histogram

```
plt.hist(df['Age'].dropna(), bins=30, edgecolor='black')
plt.title("Distribution of Passenger Ages")
plt.xlabel("Age")
plt.ylabel("Number of Passengers")
plt.show()
```



d) Scatter plot

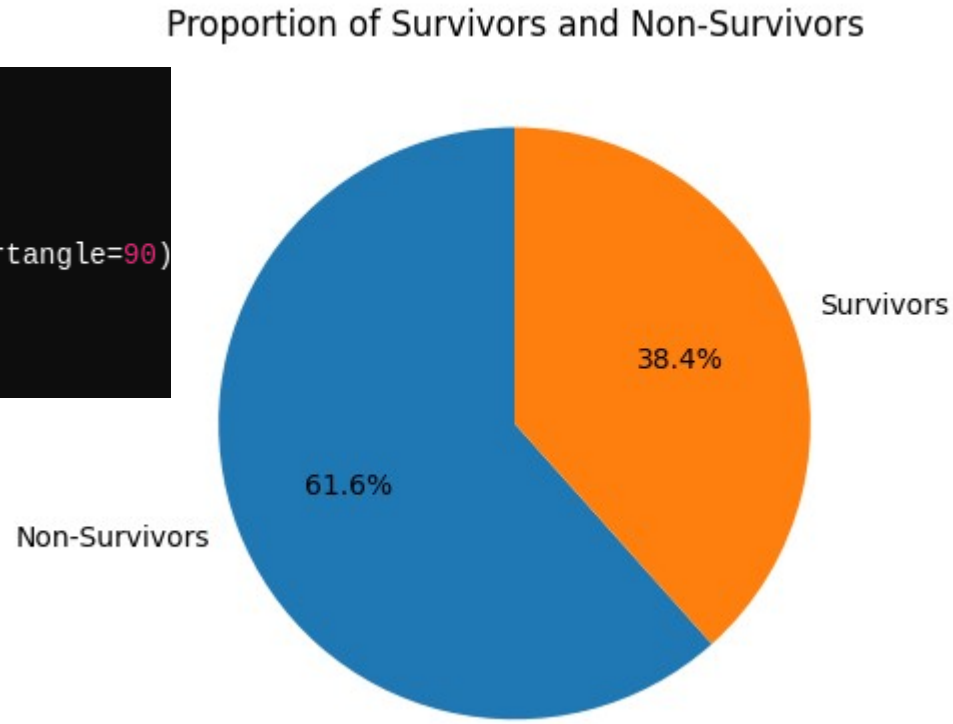
```
plt.scatter(df['Age'], df['Fare'], alpha=0.5)
plt.title("Fare vs Age of Passengers")
plt.xlabel("Age")
plt.ylabel("Fare")
plt.show()
```



e) pie chart

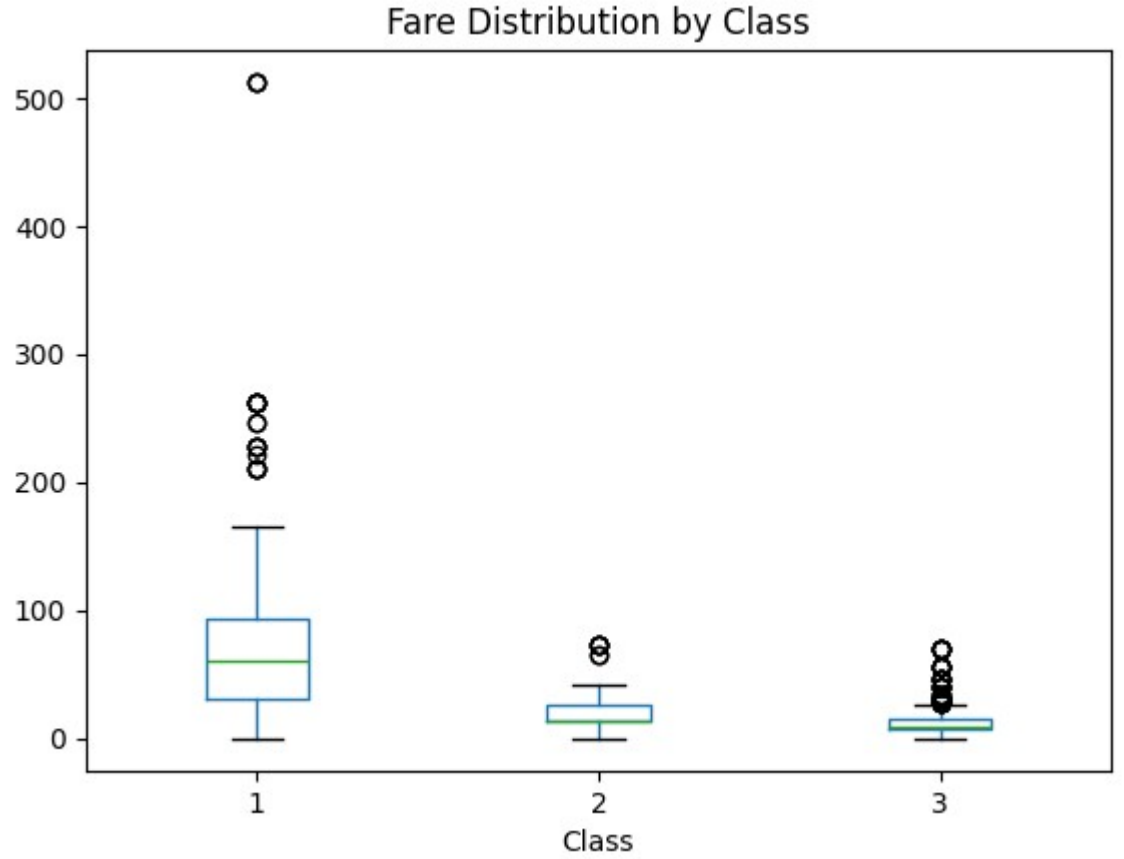
```
survived_counts = df['Survived'].value_counts()
labels = ['Non-Survivors', 'Survivors']

plt.pie(survived_counts, labels=labels, autopct='%1.1f%%', startangle=90)
plt.title("Proportion of Survivors and Non-Survivors")
plt.show()
```



f) Boîte à moustache

```
df.boxplot(column='Fare', by='Pclass', grid=False)
plt.title("Fare Distribution by Class")
plt.suptitle("")  # Suppress the automatic 'Boxplot' ti
plt.xlabel("Class")
plt.ylabel("Fare")
plt.show()
```



g) Heatmap

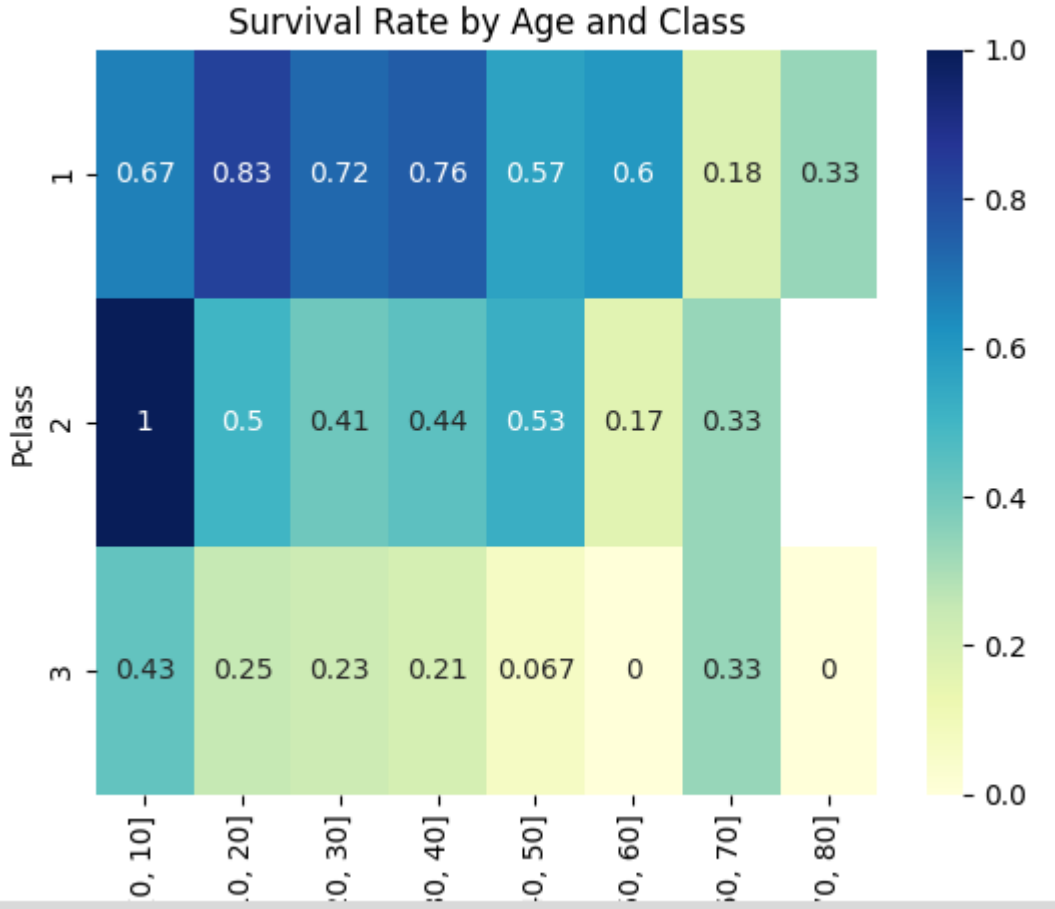
- Besoin de Seaborn pour des v

```
import seaborn as sns

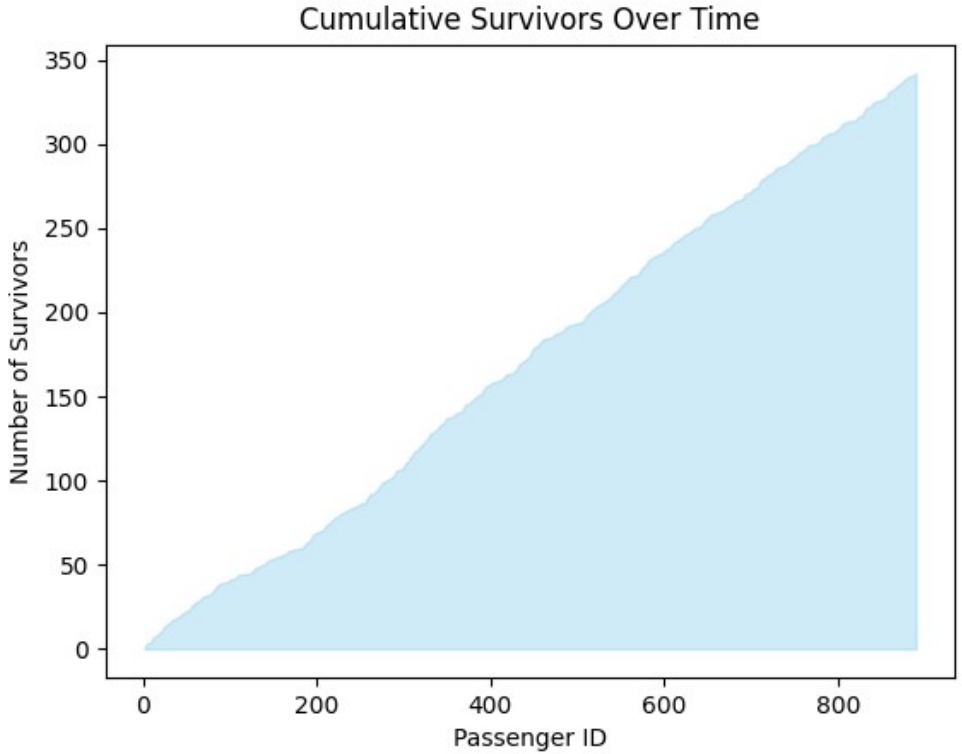
# Create bins for age
df['AgeBin'] = pd.cut(df['Age'], bins=[0, 10, 20, 30, 40, 50, 60, 70,

# Pivot table for survival rates
pivot = df.pivot_table('Survived', index='Pclass', columns='AgeBin')

# Plot heatmap
sns.heatmap(pivot, annot=True, cmap="YlGnBu")
plt.title("Survival Rate by Age and Class")
plt.show()
```

h) Area plot




```
cumulative_survivors = df[df['Survived'] == 1].groupby('PassengerId').count().cumsum()

plt.fill_between(cumulative_survivors.index, cumulative_survivors['Survived'], color="skyblue", alpha=0.4)
plt.title("Cumulative Survivors Over Time")
plt.xlabel("Passenger ID")
plt.ylabel("Number of Survivors")
plt.show()
```

4) Pour aller plus loin avec la viz ...

- Explorer de nouveaux types de graphes (violin...)
- Explorer les graphes dynamiques avec plotly
- Explorer les graphes 3D
- Explorer les graphes géographiques avec geojson, geopandas...
- Plusieurs courbes sur la même figure vs figures différentes

```python
import pandas as pd
import matplotlib.pyplot as plt

# Charger les données Titanic
url = "https://github.com/datasciencedojo/datasets/raw/master/titanic.csv"
df = pd.read_csv(url)

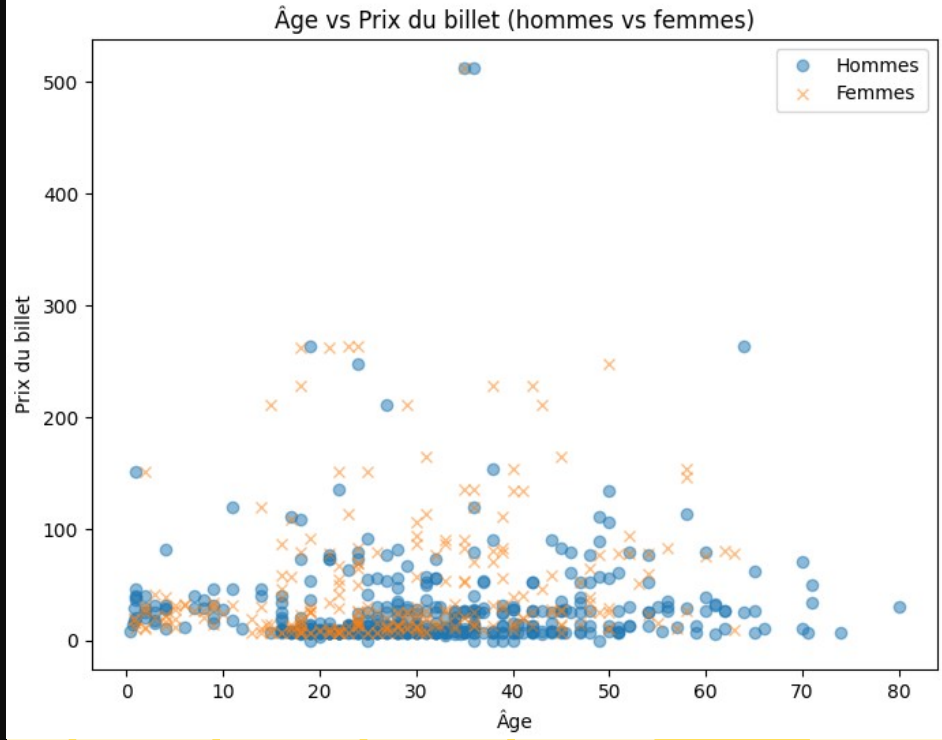
# Filtrer les passagers par sexe
male = df[df['Sex'] == 'male']
female = df[df['Sex'] == 'female']

# Créer une figure
plt.figure(figsize=(8, 6))

# Tracer 4 graphiques sur la même figure (superposition)
plt.plot(male['Age'], male['Fare'], 'o', label="Hommes", alpha=0.5)
plt.plot(female['Age'], female['Fare'], 'x', label="Femmes", alpha=0.5)

# Ajouter des légendes, titres et axes
plt.title("Âge vs Prix du billet (hommes vs femmes)")
plt.xlabel("Âge")
plt.ylabel("Prix du billet")
plt.legend()

# Afficher
plt.show()
```



```python
# Créer une figure et des sous-graphiques
fig, axs = plt.subplots(2, 2, figsize=(10, 8))  # 2 lignes, 2 colonnes de subplots

# Premier subplot
axs[0, 0].scatter(male['Age'], male['Fare'], color='blue', alpha=0.5)
axs[0, 0].set_title("Hommes - Âge vs Prix")
axs[0, 0].set_xlabel("Âge")
axs[0, 0].set_ylabel("Prix du billet")

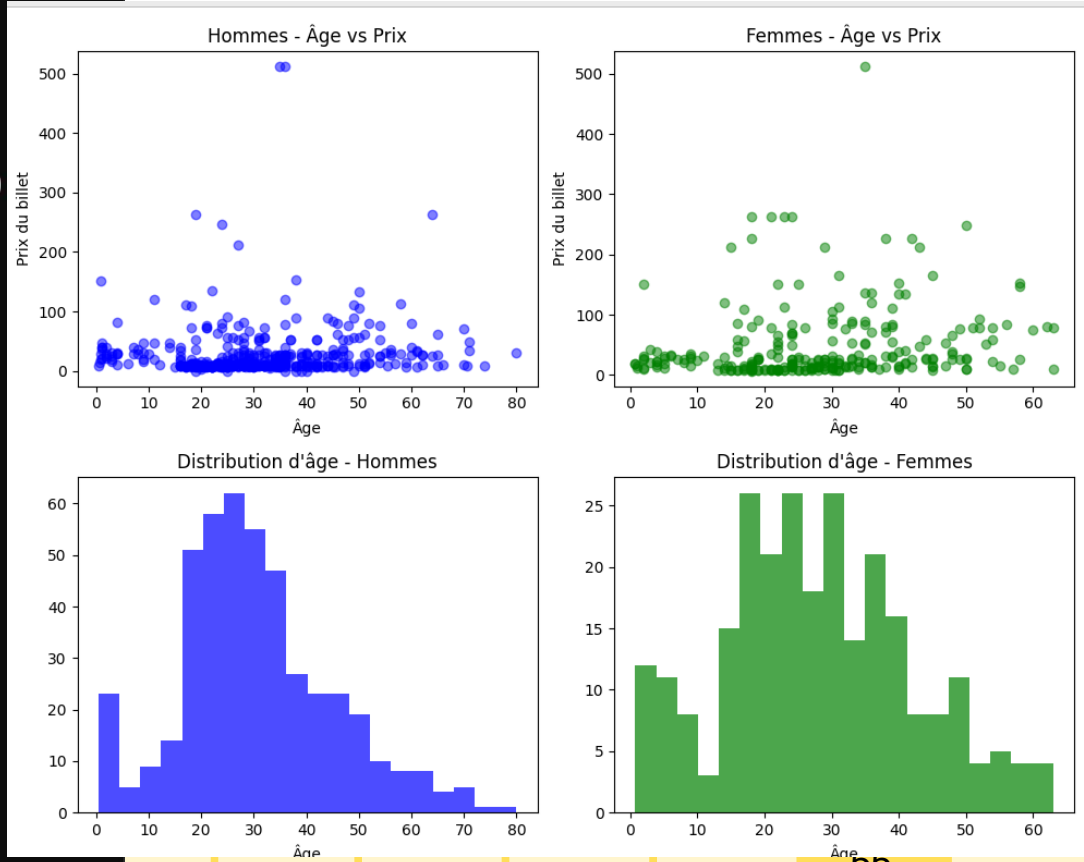
# Deuxième subplot
axs[0, 1].scatter(female['Age'], female['Fare'], color='green', alpha=0.5)
axs[0, 1].set_title("Femmes - Âge vs Prix")
axs[0, 1].set_xlabel("Âge")
axs[0, 1].set_ylabel("Prix du billet")

# Troisième subplot
axs[1, 0].hist(male['Age'].dropna(), bins=20, color='blue', alpha=0.7)
axs[1, 0].set_title("Distribution d'âge - Hommes")
axs[1, 0].set_xlabel("Âge")

# Quatrième subplot
axs[1, 1].hist(female['Age'].dropna(), bins=20, color='green', alpha=0.7)
axs[1, 1].set_title("Distribution d'âge - Femmes")
axs[1, 1].set_xlabel("Âge")

# Ajuster l'espacement entre les subplots
plt.tight_layout()

# Afficher les graphiques
plt.show()
```



De nombreuses ressources en ligne

- https://stackabuse.s3.amazonaws.com/files/ebooks/data-visualization-in-python/data-visualization-in-python-preview.pdf
- https://www.datacamp.com/tutorial/types-to-create-them-in-python
- Realpython
- ...

